# MODUL 7 SYSTEM DEVELOPMENT I

## A. Identitas Modul

| | |
|---|---|
| **Mata Kuliah** | Praktikum Sistem Informasi |
| **Modul** | 7 |
| **Topik** | System Development I |
| **Metode Pembelajaran** | Project-Based Learning (PjBL) |
| **Bentuk Proyek** | Individu |
| **Alokasi Waktu** | 1 kali tatap muka |
| **Produk Akhir Modul** | Functional Core System |

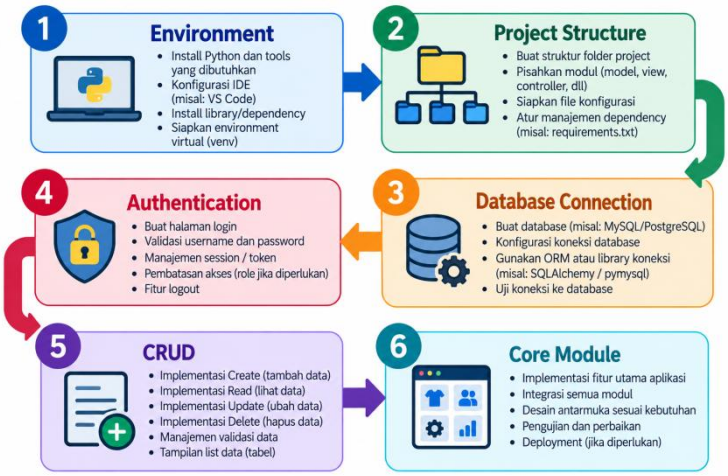
## B. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Menyiapkan environment pengembangan sistem.
2. Membuat struktur project sesuai teknologi yang digunakan.
3. Menghubungkan aplikasi dengan database.
4. Membuat mekanisme authentication.
5. Menerapkan konsep CRUD.
6. Mengimplementasikan database yang telah dirancang pada Modul 5.
7. Mengimplementasikan UI yang telah dirancang pada Modul 6.
8. Mengembangkan minimal satu modul utama sistem.
9. Melakukan validasi input pada aplikasi.
10. Menguji fungsi dasar aplikasi secara mandiri.

## C. Capaian Modul

Pada akhir modul ini, mahasiswa harus memiliki aplikasi yang sudah dapat dijalankan dan memiliki fondasi berikut:



Produk minimal:

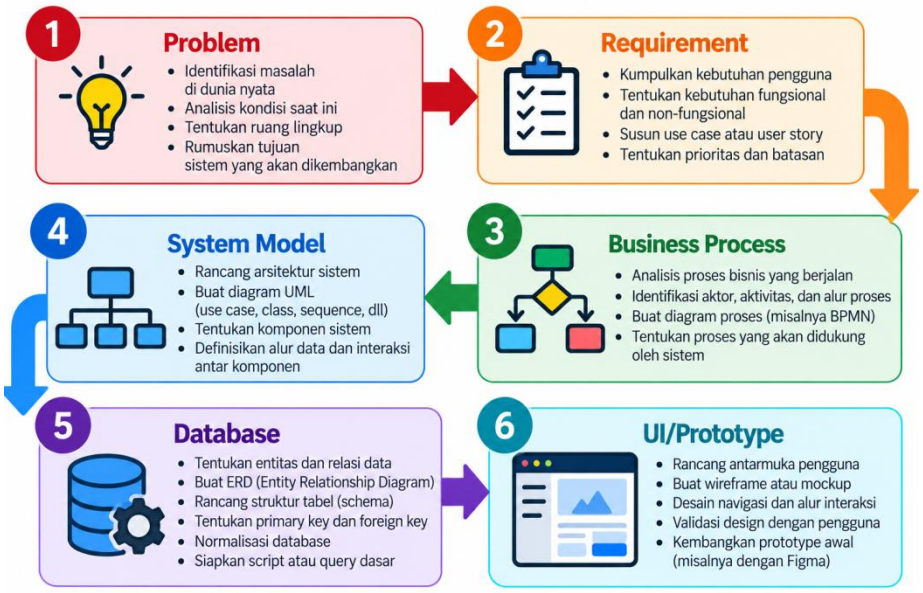
**Functional Core System**

Artinya, aplikasi bukan lagi sekadar prototype visual. Minimal satu fungsi utama harus benar-benar bekerja dan terhubung dengan database.

# D. Pendahuluan

Enam modul sebelumnya menghasilkan rancangan.

Mahasiswa telah memiliki:




Sekarang rancangan tersebut harus diterjemahkan menjadi aplikasi nyata.

Tahap ini sering menjadi titik ketika mahasiswa mulai melakukan kesalahan.

- Ada mahasiswa yang langsung membuka editor kode dan mulai membuat halaman.
- Ada yang langsung membuat database baru tanpa mengacu pada ERD.
- Ada yang mengambil template aplikasi dari internet lalu mencoba menyesuaikannya dengan proyek.
- Ada pula yang membuat fitur berdasarkan apa yang mudah dikoding, bukan berdasarkan kebutuhan sistem.

Praktikum ini menggunakan pendekatan berbeda.

**Kode harus mengikuti desain, bukan desain mengikuti kode.**

Mahasiswa harus membangun aplikasi berdasarkan artefak yang telah dibuat sebelumnya.

# E. Development sebagai Implementasi Desain

Pada tahap ini mahasiswa mengubah: **Use Case m**enjadi **Function**

mengubah: **ERD** menjadi **Database**

dan mengubah: **Prototype** menjadi **User Interface**

Secara sederhana:

**System Model → Code**

**Database Design → Database**

**UI Prototype → Interface**

Ketiganya kemudian diintegrasikan menjadi aplikasi.

# F. Environment Development

Environment adalah lingkungan yang digunakan untuk membangun dan menjalankan aplikasi.

Contoh komponen:

- Operating System;
- code editor;
- programming language;
- framework;
- web server;
- database server;
- DBMS;
- browser;
- version control.

Contoh stack:

**PHP + Laravel + MySQL**

atau:

**Python + Flask/Django + MySQL**

atau:

**Node.js + Express + MySQL**

atau teknologi lain yang telah ditentukan dalam praktikum.

Yang penting adalah prinsipnya:

Teknologi merupakan alat untuk membangun sistem, bukan tujuan utama praktikum.

Mahasiswa harus dapat menjelaskan mengapa teknologi tersebut digunakan.

# G. Project Structure

Project yang baik memiliki struktur yang jelas.

Contoh sederhana aplikasi web:

```
project/
│
├── public/
├── src/
│   ├── controllers/
│   ├── models/
│   ├── services/
│   └── views/
│
├── database/
├── routes/
├── assets/
└── config/
```

Struktur sebenarnya bergantung pada framework.

Mahasiswa tidak perlu menggunakan struktur tersebut secara persis.

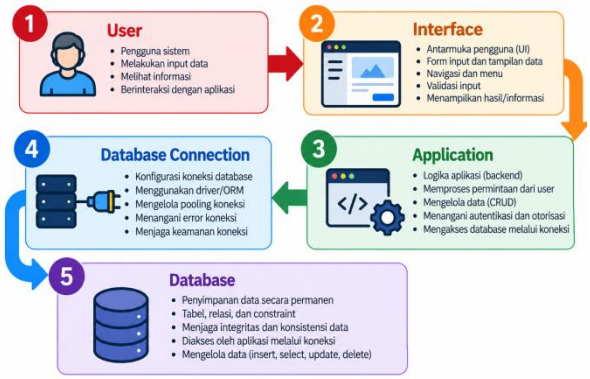
Prinsip yang harus dipahami:

**Kode harus terorganisasi agar mudah dipahami, diuji, dan dikembangkan.**

Hindari meletakkan seluruh kode aplikasi dalam satu file.

# H. Database Connection

Aplikasi harus dapat berkomunikasi dengan database. Secara konseptual:



Ketika pengguna meminta data pelanggan, aplikasi mengambil data dari database.

Ketika pengguna menyimpan transaksi, aplikasi mengirim data ke database.

Database connection merupakan fondasi seluruh fungsi CRUD.

# I. Configuration

Informasi koneksi database sebaiknya tidak ditulis sembarangan di dalam kode.

Contoh informasi:

- host;
- port;
- database name;
- username;
- password.

Gunakan konfigurasi yang sesuai dengan framework.

Untuk proyek mahasiswa, penting untuk mulai mengenalkan konsep:

**Configuration ≠ Business Logic**

Konfigurasi sebaiknya dipisahkan dari logika aplikasi.

# J. Authentication

Authentication adalah proses memastikan identitas pengguna.

Contoh:



Sistem harus mengetahui siapa pengguna yang sedang menggunakan aplikasi.

Contoh:

**Admin**

dapat mengelola semua data.

**Petugas**

dapat mengelola transaksi.

**Pelanggan**

hanya dapat melihat informasi miliknya.

Authentication berbeda dengan authorization.

# K. Authentication vs Authorization

## Authentication

Menjawab:

**Siapa Anda?**

Contoh:

Ahmad berhasil login.

## Authorization

Menjawab:

**Apa yang boleh Anda lakukan?**

Contoh:

Ahmad adalah Petugas sehingga dapat mengelola transaksi.

Keduanya penting.

Sistem yang hanya memiliki login tetapi semua pengguna dapat mengakses seluruh data belum memiliki kontrol akses yang memadai.

# L. CRUD

CRUD merupakan singkatan dari:

**Create**

**Read**

**Update**

**Delete**

CRUD merupakan operasi dasar pengelolaan data.

Contoh:

### Create

Menambahkan pelanggan.

### Read

Melihat daftar pelanggan.

### Update

Mengubah data pelanggan.

### Delete

Menghapus pelanggan.

CRUD merupakan dasar penting dalam banyak sistem informasi.

Tetapi mahasiswa harus memahami:

**CRUD bukan keseluruhan sistem informasi.**

CRUD hanya salah satu mekanisme pengelolaan data.

Sistem informasi harus memiliki proses bisnis dan aturan.

# M. Contoh CRUD Pelanggan

Misalnya entity:

**Pelanggan**

Field:

- id_pelanggan;
- nama;

- nomor_telepon;
- alamat.

Maka aplikasi memiliki:

**Create**

Form tambah pelanggan.

**Read**

Daftar pelanggan.

**Update**

Form edit pelanggan.

**Delete**

Menghapus pelanggan.

Namun penghapusan tidak boleh dilakukan tanpa mempertimbangkan relationship.

Jika pelanggan sudah memiliki transaksi, pertanyaan penting muncul:

Apakah pelanggan boleh dihapus?

Jika tidak, sistem harus memberikan perlindungan.

Misalnya:

“Pelanggan tidak dapat dihapus karena masih memiliki transaksi.”

Inilah hubungan antara database dan business rules.

# N. Validasi Input

Input pengguna tidak boleh langsung dipercaya.

Contoh:

Field:

**Nama Pelanggan**

Tidak boleh kosong.

Field:

**Nomor Telepon**

harus mengikuti format yang ditentukan.

Field:

**Jumlah**

harus berupa angka dan lebih besar dari nol.

Validasi dapat dilakukan pada:

- client side;
- server side;
- database constraint.

Untuk keamanan dan konsistensi, validasi pada server tetap penting.

# O. Feedback Sistem

Setiap operasi harus memberikan feedback.

Contoh berhasil:

Data pelanggan berhasil disimpan.

Contoh gagal:

Data pelanggan gagal disimpan.

Contoh validasi:

Nomor telepon wajib diisi.

Feedback merupakan bagian dari UX yang telah dirancang pada Modul 6.

# P. Core Module

Core module adalah modul utama yang merepresentasikan fungsi paling penting dari sistem.

Core module harus berasal dari:

**Problem + Requirement + Business Process + Use Case**

Contoh:

### Sistem Laundry

Core module:

**Transaksi Laundry**

### Sistem Laboratorium

Core module:

**Peminjaman Alat**

### Sistem Perpustakaan

Core module:

**Peminjaman Buku**

### Sistem Bengkel

Core module:

**Servis Kendaraan**

### Sistem Koperasi

Core module:

**Transaksi Simpan Pinjam**

Mahasiswa tidak boleh memilih core module hanya karena paling mudah dibuat.

Pilih berdasarkan **fungsi utama sistem**.

# Q. Implementasi Core Module

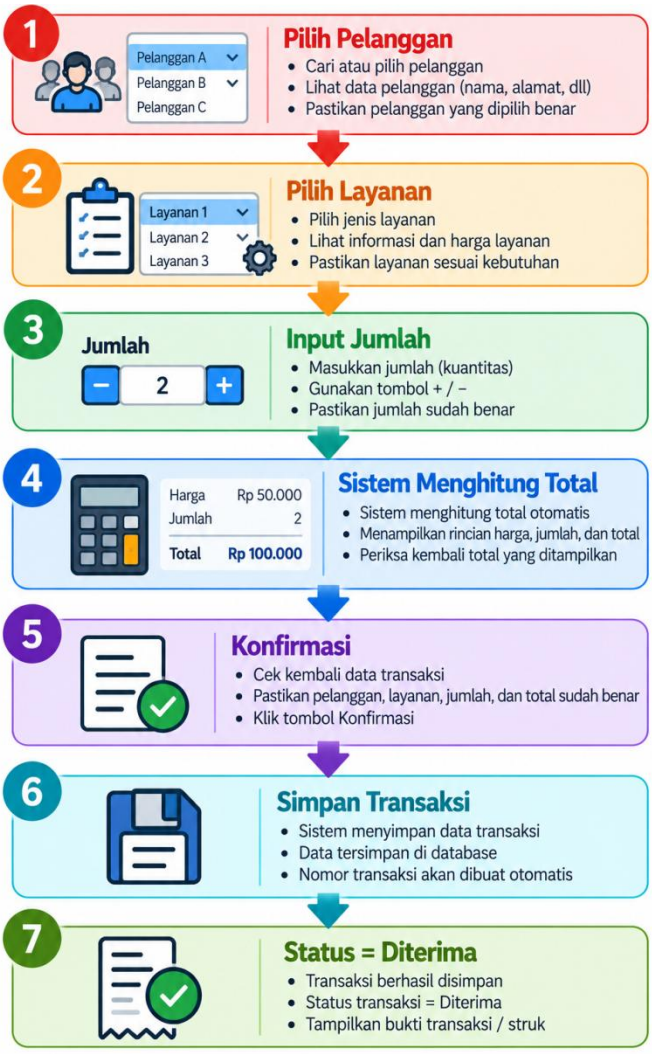
Misalnya proyek:

**Sistem Informasi Laundry**

Core module:

**Membuat Transaksi**

Alurnya:




Core module tersebut harus:

- menggunakan database;
- mengikuti use case;
- mengikuti business rules;
- menggunakan UI dari prototype;
- menghasilkan data nyata.

# R. Business Rule dalam Code

Business rule yang telah dibuat pada Modul 3 harus diterapkan.

Contoh:

**BR-01**

Pelanggan harus terdaftar sebelum transaksi dibuat.

Maka aplikasi harus memeriksa:

Apakah pelanggan tersedia?

Jika tidak:

Transaksi tidak boleh disimpan.

Contoh lain:

**BR-02**

Jumlah layanan harus lebih besar dari 0.

Maka:

jumlah = 0

harus ditolak.

Dengan demikian business rule bukan hanya dokumen.

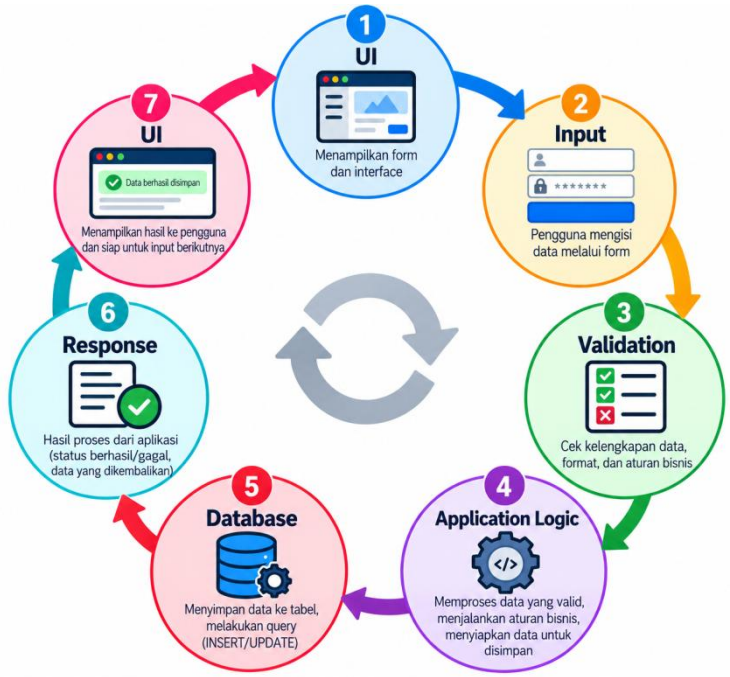
Business rule harus muncul dalam perilaku aplikasi.

# S. Integrasi UI dan Database

Prototype Modul 6 menunjukkan:

**Form Transaksi**

Tetapi pada Modul 7 form tersebut harus benar-benar bekerja. Alurnya:



Contoh:




# T. Aktivitas Praktikum

### Aktivitas 1 — Persiapkan Environment

Siapkan:

- code editor;
- bahasa pemrograman;
- framework;
- web server;
- DBMS;
- browser;
- repository Git.

Pastikan aplikasi sederhana dapat dijalankan.

# U. Aktivitas 2 — Buat Project

Buat project baru berdasarkan teknologi yang dipilih.

Gunakan nama project yang konsisten dengan proyek Modul 1.

Contoh:

```
sistem-laundry

sistem-laboratorium

sistem-perpustakaan
```

Pastikan project dapat dijalankan secara lokal.

# V. Aktivitas 3 — Hubungkan Database

Gunakan database hasil Modul 5.

Jangan membuat database baru tanpa alasan.

Hubungkan aplikasi dengan database.

Lakukan pengujian:

aplikasi dapat membaca data dari database.

Kemudian:

aplikasi dapat menyimpan data ke database.

# W. Aktivitas 4 — Implementasikan Authentication

Buat:

- halaman login;
- proses authentication;
- logout;
- session/user state;
- proteksi halaman.

Jika proyek memiliki beberapa role, implementasikan role dasar.

Contoh:

**Admin**

**Petugas**

**Pelanggan**

# X. Aktivitas 5 — Implementasikan CRUD

Pilih minimal satu entity master.

Contoh:

- pelanggan;
- barang;
- alat;
- layanan;
- anggota.

Implementasikan:

**Create → Read → Update → Delete**

Pastikan CRUD menggunakan database sebenarnya.

# Y. Aktivitas 6 — Implementasikan Validasi

Tambahkan minimal:

- required validation;
- data type validation;
- range validation;
- duplicate validation jika diperlukan.

Contoh:

Nama tidak boleh kosong.

Jumlah harus > 0.

Kode barang harus unik.

# Z. Aktivitas 7 — Implementasikan Core Module

Pilih satu use case utama dari Modul 4.

Implementasikan secara nyata.

Contoh:

**UC-03 — Membuat Transaksi**

Implementasikan seluruh alur:

**Start → Input → Validation → Processing → Database → Result**

# AA. Aktivitas 8 — Cocokkan dengan Prototype

Bandingkan aplikasi dengan prototype Modul 6.

Periksa:

- menu;
- form;
- field;
- button;
- navigation;
- feedback.

Catat perbedaan.

Tidak semua perbedaan berarti kesalahan, tetapi setiap perubahan harus memiliki alasan.

# AB. Aktivitas 9 — Git Commit

Simpan perkembangan proyek menggunakan version control.

Contoh:

```
commit 1:
initial project

commit 2:
database connection

commit 3:
authentication

commit 4:
customer CRUD

commit 5:
core transaction module
```

Commit message harus menggambarkan perubahan.

Hindari:

```
fix
update
coba
baru
final
final2
final_fix
final_beneran
```

Version control merupakan bagian dari praktik pengembangan perangkat lunak yang baik.

# AC. Lembar Kerja Mahasiswa

# LKM 1 — Development Environment

**Nama Mahasiswa:**
**NIM:**
**Nama Proyek:**

| Komponen | Teknologi | Versi | Status |
|---|---|---|---|
| OS | | | |
| Language | | | |
| Framework | | | |
| Database | | | |
| Web Server | | | |
| Code Editor | | | |
| Version Control | | | |

## LKM 2 — Database Connection

| Pengujian | Hasil | Keterangan |
|---|---|---|
| Database dapat diakses | | |
| Data dapat dibaca | | |
| Data dapat disimpan | | |
| Data dapat diperbarui | | |
| Data dapat dihapus | | |

## LKM 3 — Authentication

| Skenario | Hasil yang Diharapkan | Hasil Aktual |
|---|---|---|
| Login benar | Berhasil login | |
| Password salah | Ditolak | |
| User belum login membuka halaman protected | Ditolak | |
| Logout | Session berakhir | |

## LKM 4 — CRUD

**Entity: .....**

| Operasi | Berhasil? | Bukti |
|---|---|---|
| Create | | |
| Read | | |
| Update | | |
| Delete | | |

**LKM 5 — Core Module**

**Use Case:** .....

| Tahap | Implementasi | Status |
|---|---|---|
| Input | | |
| Validation | | |
| Processing | | |
| Database | | |
| Output | | |

# AD. Deliverable Modul 7

Setiap mahasiswa mengumpulkan:

1. Source code project.
2. Repository/version control.
3. Database terhubung.
4. Authentication.
5. Minimal satu CRUD.
6. Minimal satu core module.
7. Validasi input.
8. Bukti pengujian fungsi dasar.
9. Screenshot aplikasi.
10. Catatan perubahan dari prototype ke implementasi.

# AE. Checkpoint Dosen

Aplikasi dinyatakan **APPROVED** jika:

1. Project dapat dijalankan.
2. Database dapat terhubung.
3. Authentication berjalan.
4. CRUD berjalan.
5. Validasi input diterapkan.
6. Core module berjalan.
7. Core module menggunakan database.
8. Business rule utama telah diterapkan.
9. UI memiliki hubungan dengan prototype.
10. Source code memiliki struktur yang dapat dipahami.
11. Perkembangan proyek terdokumentasi melalui version control.

# AF. Refleksi Individu

Jawablah pertanyaan berikut:

1. Bagian implementasi apa yang paling sulit?

2. Apakah database hasil Modul 5 dapat langsung digunakan? Jika tidak, mengapa?
3. Apakah terdapat perubahan UI ketika prototype diterjemahkan menjadi aplikasi?
4. Business rule apa yang paling sulit diterapkan?
5. Bug apa yang paling sulit ditemukan?
6. Apa perbedaan antara prototype dan aplikasi yang telah berjalan?
7. Apa yang akan Anda perbaiki sebelum integrasi modul berikutnya?

# AG. Rubrik Penilaian Modul 7

| Komponen | Bobot |
|---|---|
| Environment & project structure | 10% |
| Database connection | 15% |
| Authentication | 15% |
| CRUD | 15% |
| Validation | 10% |
| Core module | 25% |
| Kesesuaian dengan desain sebelumnya | 5% |
| Version control & dokumentasi | 5% |
| **Total** | **100%** |

# AH. Tantangan Modul

Mahasiswa mungkin menemukan bahwa aplikasi yang sedang dibangun tidak persis sama dengan prototype.

Misalnya pada prototype:

**Form transaksi**

memiliki:

- pelanggan;
- layanan;
- jumlah;
- total.

Ketika mulai coding, mahasiswa menyadari bahwa satu transaksi dapat memiliki beberapa layanan.

Maka muncul kebutuhan:

**Transaksi**

dan

**Detail Transaksi**

Perubahan tersebut bukan kegagalan.

Justru hal tersebut menunjukkan bahwa proses pengembangan bersifat iteratif.

Yang penting adalah:

**Perubahan harus memiliki alasan dan tetap dapat ditelusuri ke kebutuhan sistem.**

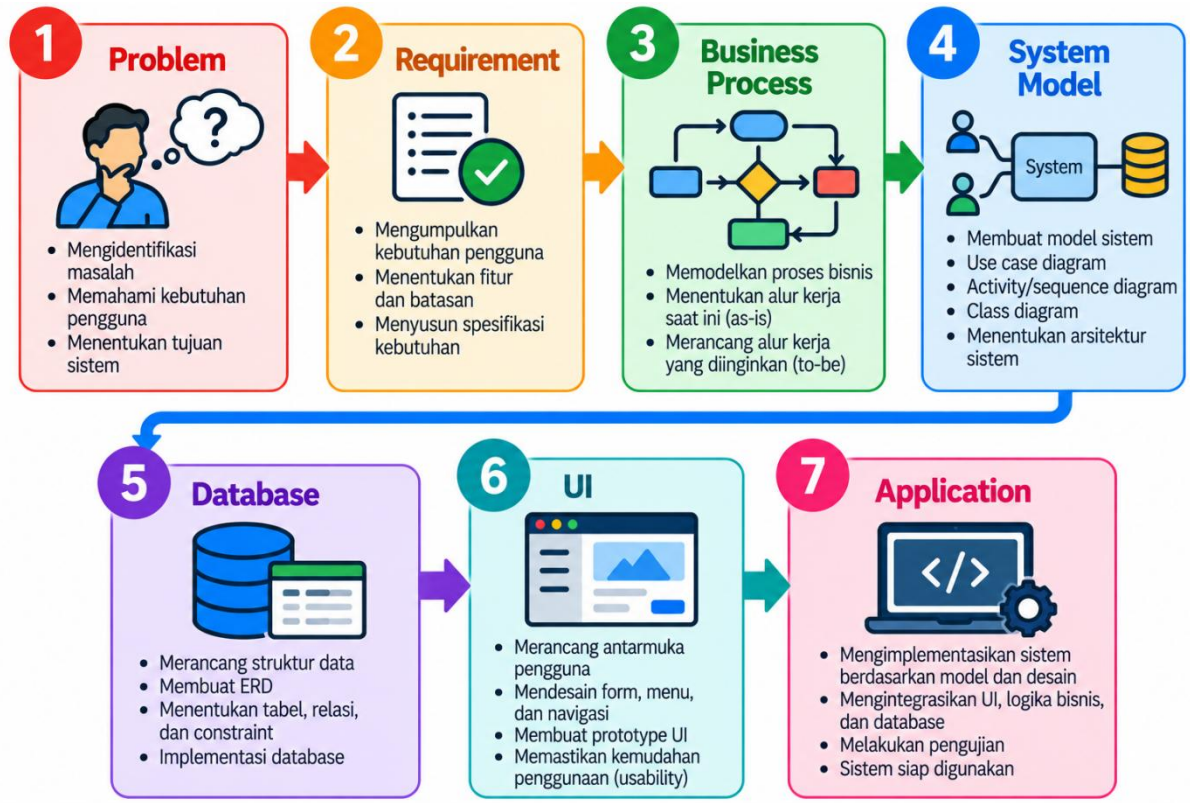
Jangan mengubah desain hanya karena:

“Lebih mudah dikoding.”

Kemudahan implementasi tidak boleh mengalahkan kebutuhan sistem.

# AI. Hubungan dengan Modul Berikutnya

Sampai Modul 7, mahasiswa sudah memiliki aplikasi yang mulai berfungsi:




Akan tetapi, aplikasi belum selesai.

Pada Modul 7 mahasiswa baru membangun fondasi dan modul utama.

Masih diperlukan:

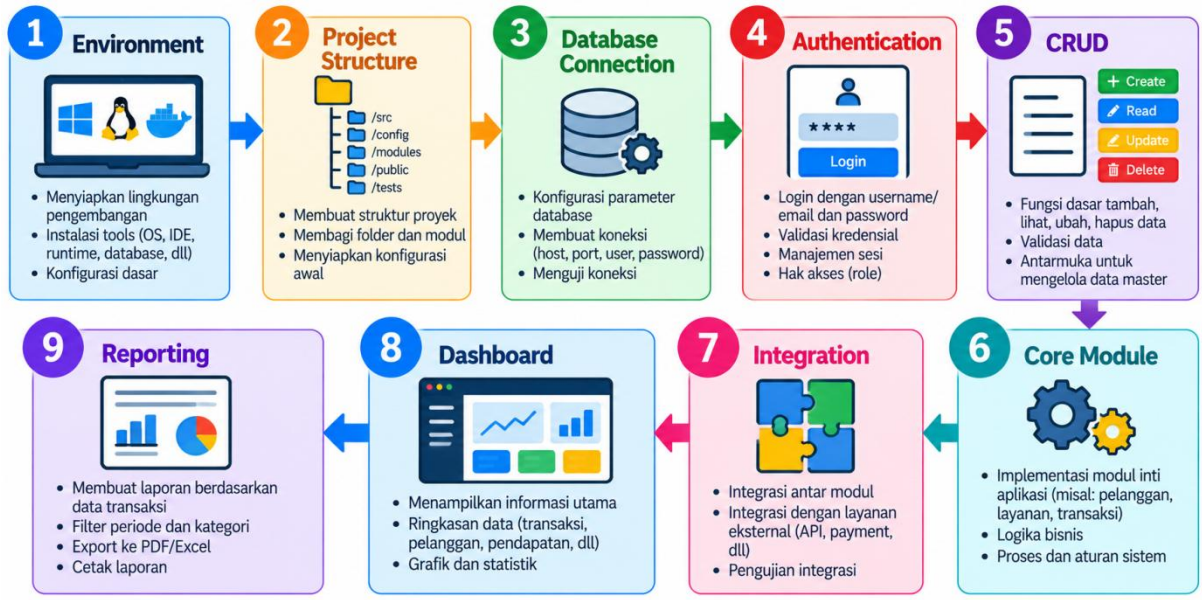
- integrasi antar-modul;
- transaksi yang lebih lengkap;
- dashboard;
- laporan;

- pencarian;
- filter;
- role/access control;
- penyempurnaan UI.

Hal tersebut akan menjadi fokus:

# MODUL 8 — SYSTEM DEVELOPMENT II

Alurnya:




Setelah Modul 8, aplikasi harus sudah menjadi **Integrated System Prototype** yang siap diuji secara sistematis pada Modul 9.